

01 - Project Charter & Design Principles

Project: Dragon Merchant Guild (*working title*)

Document Version: 1.0 (Draft)

Status: Active

1. Purpose

Dragon Merchant Guild is a long-term learning project whose primary purpose is to master C# by building a progressively more capable merchant guild simulation.

The game is **not** the end goal.

The game is the vehicle through which C# is learned.

Every programming concept learned from the curriculum should become a meaningful mechanic inside the game whenever practical.

Likewise, every game mechanic should exist for a reason beyond entertainment—it should reinforce software engineering knowledge, problem-solving, or architectural thinking.

2. Vision

Build a console-based merchant guild simulation in which dragons replace conventional machinery.

Instead of factories full of engines, conveyor belts, and robots, the world is powered by dragons with unique instincts, personalities, and natural talents.

The player manages a growing merchant guild that acquires dragons, accepts contracts, gathers resources, trains workers, expands facilities, and earns reputation across the kingdom.

Although the game begins as a console application, its architecture should naturally allow future evolution into a richer terminal interface (TUI), graphical game, or other presentation layer without requiring a complete rewrite.

3. Primary Objective

The project's success is **not measured by how complete the game becomes**.

The project's success is measured by whether the developer gains a deep understanding of modern C#, software design, and engineering practices.

Whenever gameplay and learning conflict, learning takes priority.

4. Secondary Objectives

The project should also:

- Build confidence writing medium-sized C# projects.
 - Encourage clean architecture instead of one large Program.cs.
 - Develop debugging and problem-solving habits.
 - Practice incremental refactoring.
 - Produce a portfolio-quality codebase.
 - Provide an enjoyable long-term hobby that sustains motivation.
-

5. Core Philosophy

5.1 Learn Through Building

Concepts should be learned first.

Only after understanding a concept should it be integrated into the game.

The project should never become an excuse to skip learning.

5.2 Every Lesson Earns Its Place

Each chapter of the curriculum should unlock something meaningful inside the game.

Learning should immediately create visible progress.

Examples include:

- a new mechanic

- a new dragon
- a new building
- a new system
- a guild promotion
- a major refactor

The game should visibly evolve alongside the developer's knowledge.

5.3 Build Slowly

The project intentionally favors gradual growth over rapid feature development.

Small, well-understood systems are preferred over large unfinished systems.

A finished simple feature is more valuable than five partially completed ideas.

5.4 Refactoring Is Progress

Earlier implementations are expected to improve as new concepts are learned.

Refactoring is considered part of normal development rather than a sign of failure.

Whenever a newly learned concept naturally simplifies an existing system, improving the existing code is encouraged.

5.5 Curiosity Is Encouraged

Interesting ideas should never interrupt the current learning objective.

Instead, they belong in the project backlog.

The current chapter should remain the primary focus.

6. Design Principles

6.1 Curriculum First

The curriculum determines what is implemented.

The game follows the curriculum—not the other way around.

Future-language features should not appear before their corresponding chapter unless intentionally researched.

6.2 Simplicity Before Cleverness

Readable code is preferred over impressive code.

Avoid writing code simply because it looks advanced.

6.3 Solve Today's Problem

Avoid building systems for imaginary future requirements.

New abstractions should appear only when existing code demonstrates the need.

6.4 One Improvement At A Time

A study session should introduce one primary concept.

Avoid combining multiple unrelated ideas into a single implementation.

6.5 Placeholder Assets Are Acceptable

Gameplay and code quality take priority over visuals.

Temporary ASCII art, placeholder names, or simple text menus are perfectly acceptable throughout development.

7. Project Constraints

The following constraints intentionally shape the learning experience.

Rule 1

The developer writes all production code.

AI should not generate implementation unless explicitly requested after a genuine attempt.

Rule 2

Every completed chapter must produce a meaningful improvement inside the game.

Knowledge should immediately become visible.

Rule 3

No "temporary spaghetti" with the intention of fixing everything later.

If something feels wrong, improve it while the problem is still small.

Rule 4

The game should always remain playable.

Even after major refactoring, the player should still be able to run and interact with the project.

Rule 5

Progress is measured through understanding rather than lines of code.

A day spent understanding one difficult concept is considered productive.

8. AI Philosophy

Artificial intelligence serves as a mentor.

It does not serve as a co-developer.

Its responsibilities include:

- explaining concepts
- answering questions
- reviewing code
- challenging design decisions
- identifying edge cases
- suggesting improvements
- recommending refactors

- asking "why?" before suggesting "how?"

AI should encourage reasoning instead of replacing it.

9. Development Philosophy

Development follows an iterative loop.

Study

↓

Understand

↓

Implement

↓

Test

↓

Review

↓

Refactor

↓

Play

↓

Reflect

↓

Repeat

Each loop should produce one small but meaningful improvement.

10. Definition of Success

The project succeeds if, by the end of the curriculum, the developer can confidently:

- design C# programs independently
- understand unfamiliar code
- debug problems methodically
- explain architectural decisions
- refactor confidently
- continue learning beyond the curriculum without relying on tutorials

Whether the game is "finished" is secondary.

11. Non-Goals

This project is **not** intended to:

- become a commercial game
- compete with existing factory simulators
- become a production-ready game engine
- showcase advanced graphics
- maximize feature count

Its primary role is education.

12. Guiding Question

Whenever uncertainty arises during development, ask:

"Does this decision improve my understanding of C# while making the game meaningfully better?"

If the answer is yes, it is probably the correct direction.

If the answer is no, reconsider the decision before proceeding.

13. Motto

Learn deeply. Build honestly. Improve continuously.